// Automatic FlutterFlow imports import '/backend/schema/structs/index.dart'; import '/flutter_flow/flutter_flow_theme.dart'; import '/flutter_flow/flutter_flow_util.dart'; import '/custom_code/widgets/index.dart'; // Imports other custom widgets import 'package:flutter/material.dart'; // Begin custom widget code // DO NOT REMOVE OR MODIFY THE CODE ABOVE! class ClosureReport extends StatefulWidget { const ClosureReport({ super.key, this.width, this.height, this.stflg, required this.baseUrlEnv, required this.selectedbrid, required this.customEmpId, }); final double? width; final double? height; final String? stflg; final String baseUrlEnv; final String selectedbrid; final String customEmpId; @override State<ClosureReport> createState() => _ClosureReportState(); } class _ClosureReportState extends State<ClosureReport> { @override Widget build(BuildContext context) { return Container(); } } https://uatapp.manappuram.net/AuditInspectionApi/api/GetInsp

1 message

Pranav Madhu <pranavmadhu8055@gmail.com> Tue, 3 Nov 2026 at 9:32 am
Draft

```
// Automatic FlutterFlow imports
import '/backend/schema/structs/index.dart';
import '/flutter_flow/flutter_flow_theme.dart';
import '/flutter_flow/flutter_flow_util.dart';
import '/custom_code/widgets/index.dart'; // Imports other custom widgets
import 'package:flutter/material.dart';
// Begin custom widget code
// DO NOT REMOVE OR MODIFY THE CODE ABOVE!

import 'dart:convert';
import 'package:http/http.dart' as http;

class ClosureReport extends StatefulWidget {
  const ClosureReport({
    super.key,
    this.width,
    this.height,
    this.stflg,
    required this.baseUrlEnv,
    required this.selectedbrid,
    required this.customEmpId,
  });

  final double? width;
  final double? height;
  final String? stflg;
  final String baseUrlEnv;
  final String selectedbrid;
  final String customEmpId;

  @override
  State<ClosureReport> createState() => _ClosureReportState();
}

class _ClosureReportState extends State<ClosureReport> {
  List<_AuditRow> _allRows = [];
  List<String> _auditIds = [];
  String? _selectedAuditId;
  List<_AuditRow> _displayedRows = [];

  bool _isLoading = false;
  bool _isSubmitting = false;
  String? _errorMessage;

  // Per-row inputs (same index as _displayedRows)
  List<TextEditingController> _remarkControllers = [];
  List<int> _statuses = [];

  String get _baseUrl => widget.baseUrlEnv.replaceAll(RegExp(r'/$'), '');

  void _disposeRowControllers() {
    for (final c in _remarkControllers) {
      c.dispose();
    }
    _remarkControllers = [];
```

```
    _statuses = [];
  }

  @override
  void dispose() {
    _disposeRowControllers();
    super.dispose();
  }

  @override
  void initState() {
    super.initState();
    _loadReportData();
  }

  // Fetch all report data initially
  Future<void> _loadReportData() async {
    final searchParam = widget.selectedbrid.trim();
    if (_baseUrl.isEmpty || searchParam.isEmpty) {
      setState(() => _errorMessage = 'API URL and Branch/Search parameters are required.');
      return;
    }
    setState(() {
      _isLoading = true;
      _errorMessage = null;
    });
    try {
      // Handles endpoint formats like: /Audit_id_Report/196*11-Aug-2026*9-DEC-2026/1
      final response = await http.get(Uri.parse(
        '$_baseUrl/AuditInspectionApi/api/GetInspectionData/Audit_id_Report/$searchParam/1',
      ));

      if (response.statusCode < 200 || response.statusCode >= 300) {
        throw Exception('Unable to load closure report data.');
      }

      final result = _resultList(response.body);
      if (!mounted) return;

      final rows = result
          .whereType<()
          .map(_AuditRow.fromJson)
          .toList();

      // Extract unique Audit IDs
      final ids = <String>[];
      for (final row in rows) {
        if (row.auditId.isNotEmpty && !ids.contains(row.auditId)) {
          ids.add(row.auditId);
        }
      }

      setState(() {
        _allRows = rows;
        _auditIds = ids;
        _isLoading = false;
      });
    } catch (_) {
      if (!mounted) return;
      setState(() {
        _isLoading = false;
        _errorMessage = 'Unable to load report data.';
      });
    }
  }

  // On dropdown select: filter the rows for the chosen Audit ID
  void _onAuditSelected(String auditId) {
    setState(() {
      _disposeRowControllers();
      _selectedAuditId = auditId;
      _displayedRows = _allRows.where((r) => r.auditId == auditId).toList();

      _remarkControllers = List.generate(_displayedRows.length, (i) {
        // Pre-fill with existing remarks if available from API
        return TextEditingController(text: _displayedRows[i].intAudRemark);
      });

      _statuses = List.filled(_displayedRows.length, 1);
    });
  }

  List<dynamic> _resultList(String body) {
    final decoded = jsonDecode(body);
```

```
  return decoded is Map<String, dynamic> && decoded['result'] is List
      ? decoded['result'] as List<dynamic>
      : <dynamic>[];
}

Future<void> _showAlert(String message) async {
  if (!mounted) return;
  await showDialog<void>(
    context: context,
    builder: (context) => AlertDialog(
      title: const Text('Alert'),
      content: Text(message),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(context).pop(),
          child: const Text('OK'),
        ),
      ],
    ),
  );
}

Future<void> _submitClosure() async {
  final auditId = _selectedAuditId;
  final auditorId = widget.customEmpId.trim();

  if (auditId == null) {
    await _showAlert('Please select an audit id first.');
    return;
  }
  if (_displayedRows.isEmpty) {
    await _showAlert('No audit rows to verify.');
    return;
  }
  if (auditorId.isEmpty) {
    await _showAlert('Auditor ID is required.');
    return;
  }

  // Validate remarks
  for (var i = 0; i < _displayedRows.length; i++) {
    if (_remarkControllers[i].text.trim().isEmpty) {
      await _showAlert(
        'Please enter remarks for row ${i + 1} '
        '(${_displayedRows[i].category}).',
      );
      return;
    }
  }

  setState(() {
    _isSubmitting = true;
    _errorMessage = null;
  });

  try {
    final pageValue = StringBuffer();
    final branchId = widget.selectedbrid.split('*').first; // Fallback to get ID if concatenated
    pageValue.write('${auditorId}~$branchId^');

    for (var i = 0; i < _displayedRows.length; i++) {
      final row = _displayedRows[i];
      final remarks = _remarkControllers[i].text.trim();
      final status = _statuses[i];

      // Format mapping adjusted to available JSON fields
      pageValue.write(
        '${row.auditId}~'
        '0~' // CategoryId (Placeholder since it's missing in new JSON)
        '0~' // QuestionId (Placeholder)
        '$remarks~'
        '$status~'
        '0!', // IrrCode (Placeholder)
      );
    }

    final response = await http.post(
      Uri.parse('$_baseUrl/AuditInspectionApi/api/PostInseptionCtl'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'flag': 'Audit_id_Datatable_post',
        'pageValue': pageValue.toString(),
        'paraValue': '1',
      }),
```

```
    );

    if (response.statusCode < 200 || response.statusCode >= 300) {
      throw Exception('Unable to submit audit verification.');
    }

    final decoded = jsonDecode(response.body);
    String responseMessage = 'Successfully updated status.';

    if (decoded is List && decoded.isNotEmpty && decoded.first is Map) {
      responseMessage = '${decoded.first['RES'] ?? responseMessage}';
    } else if (decoded is Map) {
      responseMessage = '${decoded['RES'] ?? responseMessage}';
    }

    if (!mounted) return;

    setState(() {
      _isSubmitting = false;
      _disposeRowControllers();
      _selectedAuditId = null;
      _displayedRows = [];
    });

    await _showAlert(responseMessage);

    if (!mounted) return;
    _loadReportData(); // Reload to get fresh data
  } catch (e) {
    if (!mounted) return;
    setState(() {
      _isSubmitting = false;
      _errorMessage = 'Unable to submit the closure action.';
    });
    await _showAlert('Unable to submit audit verification.');
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFF7F8FA),
    appBar: AppBar(
      leading: IconButton(
        tooltip: 'Back',
        icon: const Icon(Icons.arrow_back, color: Color(0xFF1A202C)),
        onPressed: () => context.go('/menuPage'),
      ),
      centerTitle: true,
      title: const Text(
        'Closure Report',
        style: TextStyle(
          fontWeight: FontWeight.w600,
          fontSize: 20,
          color: Color(0xFF1A202C),
        ),
      ),
      backgroundColor: Colors.white,
      surfaceTintColor: Colors.transparent,
      shadowColor: Colors.black12,
    ),
    body: Row(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Expanded(
          child: SingleChildScrollView(
            padding: const EdgeInsets.symmetric(
              horizontal: 48.0,
              vertical: 32.0,
            ),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.stretch,
              children: [
                if (_errorMessage != null)
                  Padding(
                    padding: const EdgeInsets.only(bottom: 16.0),
                    child: Text(
                      _errorMessage!,
                      style: TextStyle(
                        color: Colors.red.shade700,
                        fontWeight: FontWeight.w600,
                      ),
                    ),
                  ),
```

```
                _buildAuditSelectionSection(),
                if (_isLoading)
                  const Padding(
                    padding: EdgeInsets.symmetric(vertical: 40),
                    child: Center(
                      child: CircularProgressIndicator(
                        color: Color(0xFF005FB8),
                      ),
                    ),
                  ),
                if (_selectedAuditId != null && !_isLoading) ...[
                  const SizedBox(height: 32),
                  _buildAuditTableSection(),
                  const SizedBox(height: 24),
                  _buildActionButtons(),
                  const SizedBox(height: 32),
                ],
              ],
            ),
          ),
        ),
      ],
    ),
  );
}

Widget _buildAuditSelectionSection() {
  return _SectionContainer(
    title: 'Audit Selection',
    icon: Icons.assignment_ind_outlined,
    child: Row(
      mainAxisAlignment: MainAxisAlignment.start,
      children: [
        _FormDropdown<String>(
          label: 'Select Audit ID',
          width: 260,
          value: _selectedAuditId,
          items: _auditIds,
          onChanged: _isLoading
              ? null
              : (value) {
                  if (value != null) _onAuditSelected(value);
                },
          displayText: (item) => item,
          hintText: _isLoading ? 'Loading...' : 'Select...',
        ),
      ],
    ),
  );
}

Widget _buildAuditTableSection() {
  const headStyle = TextStyle(
    fontSize: 12,
    fontWeight: FontWeight.w700,
    color: Color(0xFF2D3748),
  );
  const cellStyle = TextStyle(fontSize: 12, color: Color(0xFF1A202C));

  Widget cell(String text, double w) => ConstrainedBox(
        constraints: BoxConstraints(maxWidth: w),
        child: Text(text.isEmpty ? '-' : text, style: cellStyle),
      );

  return _SectionContainer(
    title: 'Audit Details',
    icon: Icons.table_chart_outlined,
    child: _displayedRows.isEmpty
        ? const Padding(
            padding: EdgeInsets.symmetric(vertical: 16),
            child: Center(
              child: Text(
                'No data found for this audit id.',
                style: TextStyle(color: Color(0xFF4A5568)),
              ),
            ),
          )
        : LayoutBuilder(
            builder: (context, constraints) => SingleChildScrollView(
              scrollDirection: Axis.horizontal,
              child: ConstrainedBox(
                constraints: BoxConstraints(minWidth: constraints.maxWidth),
                child: DataTable(
                  headingRowColor:
```

```
                      WidgetStateProperty.all(const Color(0xFFF9FAFB)),
                  dataRowMinHeight: 40,
                  dataRowMaxHeight: 72,
                  columnSpacing: 24,
                  border: const TableBorder(
                    horizontalInside: BorderSide(color: Color(0xFFEDF2F7)),
                  ),
                  columns: const [
                    DataColumn(label: Text('Sr.', style: headStyle)),
                    DataColumn(label: Text('Category', style: headStyle)),
                    DataColumn(label: Text('Question', style: headStyle)),
                    DataColumn(label: Text('Audit Remark', style: headStyle)),
                    DataColumn(label: Text('BH Answer', style: headStyle)),
                    DataColumn(label: Text('Current Status', style: headStyle)),
                    DataColumn(label: Text('Verification Status', style: headStyle)),
                    DataColumn(label: Text('Auditor Remarks', style: headStyle)),
                  ],
                  rows: [
                    for (var i = 0; i < _displayedRows.length; i++)
                      DataRow(cells: [
                        DataCell(Text('${i + 1}', style: cellStyle)),
                        DataCell(cell(_displayedRows[i].category, 140)),
                        DataCell(cell(_displayedRows[i].question, 280)),
                        DataCell(cell(_displayedRows[i].auditorRemark, 160)),
                        DataCell(cell(_displayedRows[i].bhAnswer, 140)),
                        DataCell(cell(_displayedRows[i].intAudStatusDesc, 140)),
                        DataCell(_statusDropdown(i)),
                        DataCell(_remarkField(i)),
                      ]),
                  ],
                ),
              ),
            ),
          ),
        ),
    );
  }

  Widget _remarkField(int i) {
    return SizedBox(
      width: 220,
      height: 36,
      child: TextField(
        controller: _remarkControllers[i],
        style: const TextStyle(fontSize: 12, color: Colors.black),
        decoration: InputDecoration(
          hintText: 'Enter remarks...',
          hintStyle: const TextStyle(fontSize: 12, color: Colors.grey),
          filled: true,
          fillColor: Colors.white,
          contentPadding: const EdgeInsets.symmetric(horizontal: 10),
          border: OutlineInputBorder(
            borderRadius: BorderRadius.circular(6),
            borderSide: const BorderSide(color: Color(0xFFCBD5E0)),
          ),
          enabledBorder: OutlineInputBorder(
            borderRadius: BorderRadius.circular(6),
            borderSide: const BorderSide(color: Color(0xFFCBD5E0)),
          ),
          focusedBorder: OutlineInputBorder(
            borderRadius: BorderRadius.circular(6),
            borderSide: const BorderSide(color: Color(0xFF005FB8), width: 1.5),
          ),
        ),
      ),
    );
  }

  Widget _statusDropdown(int i) {
    return Container(
      width: 180,
      height: 36,
      padding: const EdgeInsets.symmetric(horizontal: 10),
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(6),
        border: Border.all(color: const Color(0xFFCBD5E0)),
      ),
      child: DropdownButtonHideUnderline(
        child: DropdownButton<int>(
          value: _statuses[i],
          isExpanded: true,
          icon: const Icon(Icons.arrow_drop_down, color: Color(0xFF4A5568)),
          style: const TextStyle(fontSize: 12, color: Color(0xFF1A202C)),
          items: const [
```

```
            DropdownMenuItem(value: 1, child: Text('Satisfactory closed')),
            DropdownMenuItem(value: 2, child: Text('Wrong Rectification')),
          ],
          onChanged: (value) {
            if (value != null) setState(() => _statuses[i] = value);
          },
        ),
      ),
    );
  }

  Widget _buildActionButtons() {
    return Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        ElevatedButton.icon(
          onPressed: _isSubmitting ? null : () => _submitClosure(),
          icon: _isSubmitting
              ? const SizedBox(
                  width: 16,
                  height: 16,
                  child: CircularProgressIndicator(
                    color: Colors.white,
                    strokeWidth: 2,
                  ),
                )
              : const Icon(Icons.check),
          label: Text(_isSubmitting ? 'Submitting...' : 'Confirm Validation'),
          style: ElevatedButton.styleFrom(
            backgroundColor: const Color(0xFF16803C),
            foregroundColor: Colors.white,
            padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(6),
            ),
          ),
        ),
        const SizedBox(width: 16),
        OutlinedButton.icon(
          onPressed: _isSubmitting ? null : () => context.go('/menuPage'),
          icon: const Icon(Icons.exit_to_app),
          label: const Text('Exit'),
          style: OutlinedButton.styleFrom(
            foregroundColor: const Color(0xFF2D3748),
            padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 12),
            side: const BorderSide(color: Color(0xFFCBD5E0)),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(6),
            ),
          ),
        ),
      ],
    );
  }
}

// --- Reusable UI Components
class _SectionContainer extends StatelessWidget {
  final String title;
  final IconData icon;
  final Widget child;
  final EdgeInsetsGeometry padding;
  const _SectionContainer({
    required this.title,
    required this.icon,
    required this.child,
    this.padding = const EdgeInsets.all(24.0),
  });
  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withOpacity(0.05),
            blurRadius: 20,
            offset: const Offset(0, 5),
          ),
        ],
        border: Border.all(color: const Color(0xFFEDF2F7)),
      ),
      child: Column(
```

```
        children: [
          const SizedBox(height: 8),
          Container(
            padding: const EdgeInsets.symmetric(
              horizontal: 24.0,
              vertical: 16.0,
            ),
            decoration: const BoxDecoration(
              color: Color(0xFFF9FAFB),
              borderRadius: BorderRadius.only(
                topLeft: Radius.circular(12),
                topRight: Radius.circular(12),
              ),
              border: Border(
                bottom: BorderSide(color: Color(0xFFEDF2F7)),
              ),
            ),
            child: Row(
              children: [
                Icon(
                  icon,
                  size: 22,
                  color: const Color(0xFF4A5568),
                ),
                const SizedBox(width: 12),
                Text(
                  title,
                  style: const TextStyle(
                    fontSize: 16,
                    fontWeight: FontWeight.w600,
                    color: Color(0xFF2D3748),
                  ),
                ),
              ],
            ),
          ),
          Padding(
            padding: padding,
            child: child,
          ),
        ],
      ),
    );
  }
}

class _FormDropdown<T> extends StatelessWidget {
  final String label;
  final double width;
  final T? value;
  final List<T> items;
  final void Function(T?)? onChanged;
  final String Function(T) displayText;
  final String hintText;
  const _FormDropdown({
    Key? key,
    required this.label,
    required this.width,
    required this.value,
    required this.items,
    required this.onChanged,
    required this.displayText,
    this.hintText = 'Select...',
  }) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return SizedBox(
      width: width,
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            label,
            style: const TextStyle(
              fontSize: 11,
              fontWeight: FontWeight.w600,
              color: Color(0xFF4A5568),
            ),
          ),
          const SizedBox(height: 8),
          Container(
            height: 36,
            padding: const EdgeInsets.symmetric(horizontal: 10),
            decoration: BoxDecoration(
```

```
              color: Colors.white,
              borderRadius: BorderRadius.circular(6),
              border: Border.all(color: const Color(0xFFCBD5E0)),
              boxShadow: [
                BoxShadow(
                  color: Colors.black.withOpacity(0.02),
                  blurRadius: 4,
                  offset: const Offset(0, 2),
                ),
              ],
            ),
            child: DropdownButtonHideUnderline(
              child: DropdownButton<T>(
                hint: Text(
                  hintText,
                  style: const TextStyle(fontSize: 12, color: Colors.grey),
                ),
                value: value,
                isExpanded: true,
                icon:
                    const Icon(Icons.arrow_drop_down, color: Color(0xFF4A5568)),
                items: items.map((item) {
                  return DropdownMenuItem<T>(
                    value: item,
                    child: Text(
                      displayText(item),
                      style: const TextStyle(fontSize: 12),
                      overflow: TextOverflow.ellipsis,
                    ),
                  );
                }).toList(),
                onChanged: onChanged,
                style: const TextStyle(
                  fontSize: 12,
                  color: Color(0xFF1A202C),
                ),
              ),
            ),
          ),
        ],
      ),
    );
  }
}

// --- Data Models
class _AuditRow {
  const _AuditRow({
    required this.auditId,
    required this.category,
    required this.question,
    required this.auditorRemark,
    required this.bhAnswer,
    required this.intAudRemark,
    required this.intAudStatusDesc,
  });

  final String auditId;
  final String category;
  final String question;
  final String auditorRemark;
  final String bhAnswer;
  final String intAudRemark;
  final String intAudStatusDesc;

  factory _AuditRow.fromJson(Map<String, dynamic> json) => _AuditRow(
        auditId: '${json['AUDIT_ID'] ?? ''}',
        category: '${json['CATEGORY'] ?? ''}',
        question: '${json['QUESTION'] ?? ''}',
        auditorRemark: '${json['AUDITOR_REMARK'] ?? ''}',
        bhAnswer: '${json['BH_ANSWER'] ?? ''}',
        intAudRemark: json['INT_AUD_REMARK'] != null ? '${json['INT_AUD_REMARK']}' : '',
        intAudStatusDesc: '${json['INT_AUD_STATUS_DESC'] ?? ''}',
      );
}
```